# Glass Collections and Gates

## Core Concepts

**Opcodes**:  The entry point. An opcode identifies which root collection decodes the payload; conceptually it is the top-level gate: "this opcode type → decode this collection, starting at bit 0."  A top-level gate is just a gate that has no parent in this context.  Opcodes are maintained by the Opcode Editor.

**Collection**: A named, ordered list of field definitions.  A flat record layout.  Arrays and nesting are never expressed inside of a collection. Collections are maintained by the Collection Editor.

**Field**: A named value in a collection.  Fields have an encoding that tells us how to interpret their bits. They have length and an offset into the collection.  Fields can hold values like integers, floating point numbers, strings.  They  can hold aggregations like BLOBs or Gates that represent other collections.  Fields are maintained by the Collection Editor.

**Gate**: A field type belonging to one collection that identifies a child collection.  Gates define multiplicity rules (Once, Times, UntilEnd).  Gates are maintained by the Gate Editor.

**Predicate**: A per-field presence condition evaluated against another field's value.  The source field may be in the same collection (bare name) or an ancestor collection (qualified <collection>.<field> as name).  Fields have a position in the collection.  Fields that are not present take up 0 space but still have a position, thus other fields can be located relative to a predicated field.

**Relative Field**: A field whose offset is measured from the end of another (anchor) field instead of from the start of a collection.  The bit-offset of a relative field is added to the end of this other anchor field to specify the location of the relative field.  This is required after variable-length fields (strings, gates) since we have no way of knowing the absolute offset in advance.

## Multiplicity:  Choosing the Gate Kind

Multiplicity determines how many times a child collection appears in the bit stream. There are three different types available.

Once: the child collection appears exactly one time at this position.  Use for a nested field structure.

Times: The child repeats N times.  N comes from either a integer field that is evaluated at runtime, or a constant that is specified in the gate.  If a count field is used, it must have been seen prior to the gate that uses it.

UntilEnd: The child repeats until the payload is exhausted.  Use only when the repeating records run to the end of the packet with nothing after them.  Arrays of structures in a message for example.  Tracking, MovementHistory are good examples of top-level UntilEnd multiplicities.  If a partial record is found at the end, the extraction is rolled back and the partial record is discarded.  The bits of the rolled back region can then be matched against other fields.

## Predicates: Conditional Field Presence

A predicate may optionally be added to a field and will decide if the field is present in this instance (thus the field is optional).  The field is decoded only when the predicate holds, otherwise the field is skipped and consumes no payload.

<sourceField> <operator> <operand>

A predicate defines a source field which must decode to an integer.  Bitmask tests require an unsigned source.  **Currently source fields must be in the same collection**.

The predicate expression uses binary operators, and the source field must appear on the left.  A constant appears on the right.

Operators available:

| | |
|---|---|
| == | Equal |
| != | Not equal |
| > >= | Greater than, greater than or equal to |
| < <= | Less than, less than or equal to. |
| & | bitmask |

Constants may be specified in decimal (5), negative decimal (-1), or hex (0x1F). Whitespace around parts is optional.

Examples: Flags&0x04, ItemClass == 1, Count > 0.

# Helpful Patterns

These are the shapes that come up over and over again when modeling messages. It is helpful to recognize which pattern applies before creating any rows in the database.

## The Array of Structures

**Use this**: When the payload contains a run of identical records.

Model this with a collection describing the common record structure.

Create a Gate which identifies this new collection as its child. To set multiplicity you need to identify the size of the collection. If it is constant, create a Times multiplicity with the constant. If the count is in a field, then create a Times multiplicity with the fieldname identified. Or you may use “UntilEnd” if you want to consume the remainder of the payload.

You may either use the Gate in the Opcode (to identify the top-level collection), or you may create a field in another collection an add the Gate encoding (selecting your new Gate by name).

## The Prefixed Record

Sometimes a child collection is not just a single record, but has one or more prefix fields followed by a record. Inventory child items are a good example. A container may have multiple child items, so we need to create a Gate with multiplicity to consume each child. Each child is an Item collection which we can reused, but this Item is preceded by a slot index number which must be read out.

Model this using the Collection Editor as a small wrapper collection containing the prefix fields, followed by a Gate with multiplicity Once for the record we are wrapping.

In the Gate Editor, create a gate with multiplicity Once with the wrapper collection created above.

In the parent collection, at the position where the repetition begins, add a Gate Field for the wrapper collection. Set the multiplicity as required, and indicate that the wrapper collection is the child.

## The Recursive Structure

Sometimes a record needs to contain multiple instances of the same record type. One of example is a container with items inside.

We assume that you know how to end the recursion. Whether there is a child count of 0, or a presence flag that fails, recursion must terminate at some point.

Model this with a Gate using the record that serves as the top-level for recursion. The record identified by the Gate presumably will contain the recursive reference.

In the Collections editor create a field for the top level gate and add it to the containing collection. Have it reference the top-level gate.

In the GateEditor create a second gate which will be used for the recursive call. This references the same collection as the first gate.

In the Collections Editor, create a field for the recursive call inside of the recursive record. Make sure that you have some sort of predicate so that recursion does not continue forever. You might also use the multiplicity to control the number of recursive calls.